Universidad Autónoma de Madrid

Departamento de Informática

Proyecto de Sistemas Informáticos
Introducción a JavaScript

Pablo C. Cañizares
Roberto Marabini
Alejandro Bellogín

Registro de Cambios

Versión[1]	**Fecha**	**Autor**	**Descripción**
1.0	01.12.2023	PC	Primera versión.
2.0	01.10.2024	AB	Revisión y creación de snippets descargables.

[1] La asignación de versiones se realizan mediante 2 números $X.Y$. Cambios en Y indican aclaraciones, descripciones más detalladas de algún punto o traducciones. Cambios en X indican modificaciones más profundas que o bien varían el material suministrado o el contenido de la práctica.

Índice

1. Objetivo **4**

2. Introducción **4**

3. Hello world! **4**

4. Estructura del código **5**

5. Directivas de interación **7**

5.1. Alert 7

5.2. Prompt 7

5.3. Confirm 8

5.4. Console 8

6. Variables **9**

6.1. Declaración 9

6.2. Constantes 10

7. Tipos de datos **10**

7.1. Tipos básicos 11

7.1.1. Números 11

7.1.2. Cadenas 12

7.1.3. Booleanos 12

7.1.4. Otros 12

7.2. Conversiones de tipos 13

7.2.1. Conversión numérica 13

7.2.2. Conversión de cadenas 14

8. Operaciones básicas **14**

8.1. Operaciones aritméticas 14

8.2. Concatenación de cadenas 15

8.3. Modificación 'inplace' 15

8.4. Incremento/decremento 16

9. Estructuras condicionales 16

9.1. If-then-else 16

9.2. Operadores lógicos 17

10. Bucles 19

10.1. Bucle while 20

10.2. Bucle do-while 21

10.3. Bucle for 21

11. Funciones 22

11.1. Declaración y uso 22

11.2. Funciones anónimas 23

1. Objetivo

El objetivo de esta parte de la práctica consiste en obtener los conocimientos básicos del lenguaje de programación *JavaScript*. Para ello, en las diferentes subsecciones se introducen aspectos básicos – desde la sintáxis básica, declaración de variables, operaciones, funciones etc – que serán fundamentales para el entendimiento y consecución de las siguientes prácticas. Cabe destacar que, como la práctica anterior, esta práctica está basada en un tutorial guionizado (ver `https://javascript.info/first-steps`).

2. Introducción

JavaScript es un lenguaje de programación interpretado y débilmente tipado que surgió con el principal objetivo de dotar de dinamismo a las páginas web. Por ejemplo, permite añadir nuevo contenido HTML a la página, cambiar el contenido existente, reaccionar a acciones del usuario como clicks, pulsado de teclas, hacer preguntas al usuario, etc. Los programas codificados con este lenguaje son denominados `scripts`, se incrustan en el código HTML y se ejecutan automáticamente cuando la página web termina su proceso de carga de manera satisfactoria.

3. Hello world!

Procedemos a dar el primer paso, un clásico en todos los lenguajes, vamos a codificar el `HelloWorld` en *JavaScript*. Tal como se puede observar en el Listing 1, el código *JavaScript* puede incluirse directamente como parte del documento HTML, incluyendo el código entre las etiquetas `<script>`. En este caso, el ejemplo únicamente muestra por pantalla una ventana de alerta, mostrando el mítico saludo.

Listado 1: Hello world en JavaScript

```
<!DOCTYPE HTML>
<html>
<body>
```

```
  <p>Before the script...</p>

  <script>
    alert( 'Hello, world!' );
  </script>

  <p>...After the script.</p>

</body>
</html>
```

Código fuente:

4. Estructura del código

Los `scripts` de *JavaScript* pueden incluirse en el código HTML de dos maneras diferentes. Puede incrustarse directamente en el código HTML, tal como vimos en el ejemplo de la sección anterior `HelloWorld` (ver Listing 1). O por el contrario, se puede importar código *JavaScript* existente:

Listado 2: JavaScript importado desde fichero

```
// myscript.js
/**
 * Devuelve una lista de los numeros primos menores que `max`.
 */
function getPrimes(max) {
  const isPrime = Array.from({ length: max }, () => true);
  isPrime[0] = isPrime[1] = false;
  isPrime[2] = true;
  for (let i = 2; i * i < max; i++) {
    if (isPrime[i]) {
      for (let j = i ** 2; j < max; j += i) {
```

```
        isPrime[j] = false;
      }
    }
  }
  return [...isPrime.entries()]
    .filter(([, isPrime]) => isPrime)
    .map(([number]) => number);
}
```

Listado 3: HMLT importando JavaScript desde fichero

```
// Main HTML
<!DOCTYPE HTML>
<html>
<script src="myscript.js"></script>
<body>

  <p>Before the script...</p>

  <script>
    alert(getPrimes(10)); // [2, 3, 5, 7]
  </script>

  <p>...After the script.</p>

</body>
</html>
```

Códigos fuente:

Tal como podemos ver en el Listing 2, a diferencia del ejemplo `HelloWorld`, el código se encuentra en un fichero externo `myscript.js`, el cual contiene una función llamada `getPrimes`. Este archivo debe ser importado en el código HTML a través de la directiva `<script src=“ ”></script>` (ver Listing 3). Una vez importado, puede

invocarse la función definida, utilizando la etiqueta `script`.

5. Directivas de interación

Con el objetivo de proporcionar información al usuario y comunicarse con él, *JavaScript* provee de una serie de mecanismos invocables desde el código: `alert`, `prompt`, `confirm` y `console`. Estas directivas se pueden incluir en el propio código o probarlas en cualquier consola de *JavaScript*, como la disponible en los navegadores[2].

5.1. Alert

La directiva `alert`, permite mostrar en pantalla el mensaje proporcionado por el usuario:

```
alert('Hola!'); // Hola!
```

El resultado de la instrucción anterior se muestra en la siguiente figura:

Figura 1: Mensaje enviado por el navegador a través de la directiva alert

5.2. Prompt

La directiva `prompt`, permite al usuario introducir datos al sistema:

```
prompt("Por favor, introduce tu nombre", "Chuck Norris");
```

El resultado de la instrucción anterior se muestra en la siguiente figura:

[2]Por ejemplo, en Chrome, desde las *Herramientas para desarrolladores*, se pueden escribir estas u otras directivas en la pestaña denominada *Console* y confirmar su funcionamiento.

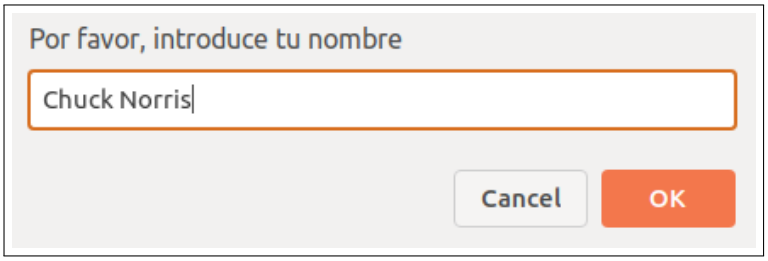

Figura 2: Mensaje mostrado por el navegador a través de la directiva prompt

5.3. Confirm

La directiva `confirm`, facilita realizar confirmaciones al usuario:

```
confirm("Por favor, confirme la operacion");
```

El resultado de la instrucción anterior se muestra en la siguiente figura:

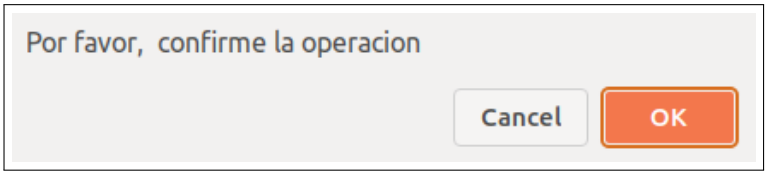

Figura 3: Mensaje mostrado por el navegador a través de la directiva confirm

5.4. Console

Otra de las funcionalidades ofrecidas por *JavaScript* es el mecanismo `Console`, que permite llevar a cabo la construcción de un log en el navegador. Además, permite definir distintos niveles de log, dependiendo de la importancia de la operación:

```
console.log('Log nivel estandar');
console.info('Log nivel informativo');
console.debug('Log nivel debug');
console.warn('Log nivel warning');
console.error('Log nivel error');
```

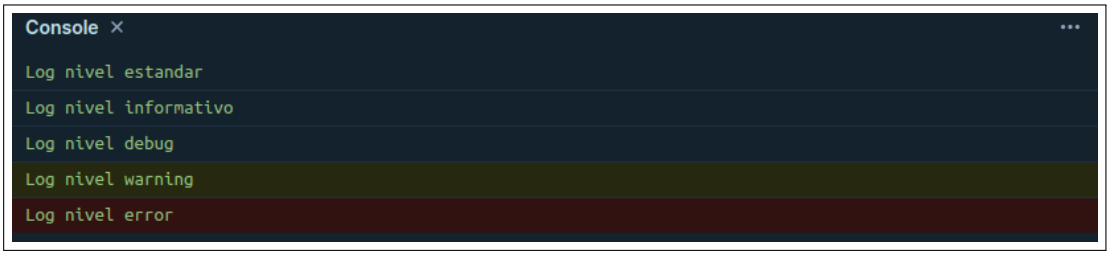

Figura 4: Mensaje mostrado por el navegador a través de la directiva

6. Variables

El uso de variables en los lenguajes de programación imperativos es una práctica habitual e indispensable para facilitar su uso y maximizar la re-usabilidad. En esta sección se introduce el uso de variables en *JavaScript*, específicamente su declaración (sección 6.1) y uso de constantes (sección 6.2)

6.1. Declaración

Para declarar una variable debemos utilizar la palabra reservada *let* seguido del nombre de la variable. Tal como se puede observar, no es necesario proporcionar el tipo de la variable a declarar. Esto es posible ya que *JavaScript* es dinámicamente tipado.

```
let mensaje;
```

Para darle valor a la variable, se utiliza el operador = seguido del valor a asignar. En esta caso se le asigna una cadena de tipo `string` y se muestra al usuario a través de la directriz `alert`:

```
let mensaje;
mensaje = 'Hola!';
alert(mensaje); // Hola!
```

Cabe destacar, que se puede inicializar la variable en la misma línea de declaración. Esto se puede hacer con una única variable:

```
let mensaje = 'Hola!';
alert(mensaje); // Hola!
```

O con múltiples variables en la misma línea:

```
let user = 'Pablo', age = 37, message = 'Hola!';
```

6.2. Constantes

Además de permitir el uso de variables, *JavaScript* facilita el uso de un tipo específico de variables, llamadas constantes, que no permiten cambios durante el tiempo de ejecución. En el siguiente ejemplo podemos ver la declaración de 4 colores distintos, asociados posteriormente a una variable y finalmente se muestra su valor a través de `alert`.

```
const COLOR_ROJO = "#F00";
const COLOR_VERDE = "#0F0";
const COLOR_AZUL = "#00F";
const COLOR_NARANJA = "#FF7F00";

// ...tomamos un color
let color = COLOR_NARANJA;
alert(color); // #FF7F00
```

7. Tipos de datos

Una vez aprendido cómo se crean variables, es importante conocer sus posibles tipos. En esta sección vamos a ver los tipos básicos (sección 7.1) y conversiones de tipos (sección 7.2).

7.1. Tipos básicos

En esta subsección, se introducen los tipos básicos en *JavaScript*. Esto es, los números, cadenas, booleanos y otros tipos. En el listado mostrado a continuación, se pueden observar todos los tipos previamente citados. Cabe destacar que la instrucción `typeof` permite conocer el tipo de una variable dada.

```
typeof undefined // "undefined"
typeof 0 // "number"
typeof 10n // "bigint"
typeof true // "boolean"
typeof "foo" // "string"
typeof Symbol("id") // "symbol"
```

7.1.1. Números

En primer lugar, se van a introducir los números. *JavaScript* permite, entre otros, el uso de números enteros y de coma flotante:

```
//Enteros
let n = 123;
alert(n)
//Coma flotante
n = 12.345;
alert(n)
```

Además, para facilitar la representación y manejo de cifras muy elevadas se introduce el tipo `BigInt`:

```
//BigInt la "n" al final de la variable denota que es de tipo bigInt
const bigInt = 1234567890123456789012345678901234567890n;
alert(bigInt)
```

7.1.2. Cadenas

Otro factor importante del lenguaje es el uso de cadenas. Para ello se debe asignar a una variable una cadena de texto, utilizando los caracteres “ ”, ’ ’ o ‘ ‘, tal y como puede observarse a continuación:

```
let str = "Hola, usando comillas dobles";
let str2 = 'Las comillas simples tambien estan permitidas';
let phrase = `se puede embeber una cadena dentro de otra ${str}`;

alert(str);
alert(str2);
alert(phrase);
```

7.1.3. Booleanos

Las variables booleanas permiten almacenar los valores `verdadero` o `falso`. Hay dos formas diferentes de asignarles un valor. En primer lugar, asignándoles directamente los valores `true` o `false`.

```
let esMayor = true;
let esMenor = false;
```

O, por otro lado, también se puede asignar a la variable booleana una expresión. En este caso, $4 > 1$, que como es verdadero, asigna `true` a la variable `esMayor`.

```
let esMayor = 4 > 1;

alert(esMayor); // true (la comparacion de este resultado es "si")
```

7.1.4. Otros

Finalmente, *JavaScript* proporciona el uso de `null` y `undefined`, que son utilizados para denotar que una variable es nula, o que no está definida. Por defecto, si

queremos que una variable sea nula debemos inicializarla como se muestra a continuación:

```
let age = null;
```

De lo contrario, las variables no inicializadas, tomarán el valor `undefined`:

```
let age;
alert(age); // shows "undefined"
```

Por último, si el usuario lo requiere, se le puede asignar el valor `undefined` a una variable:

```
let edad = 100;
// cambiar el valor a undefined
edad = undefined;
alert(edad); // "undefined"
```

7.2. Conversiones de tipos

A la hora de desarrollar un programa en cualquier lenguaje de programación, es necesario realizar conversiones de tipos de algunas variables. Como no podría ser de otra forma, *JavaScript* proporciona mecanismos para llevarlos a cabo. A continuación, se introducen las conversiones numéricas y de cadenas.

7.2.1. Conversión numérica

Para realizar la conversión de cadenas a números, basta con asignar a una variable la conversión `Number()`, tal como se muestra a continuación:

```
let str = "123";
alert(typeof str); // string

let num = Number(str); // se convierte en el numero 123
alert(typeof num); // number
```

Importante: si se intenta realizar una conversión de una cadena utilizando `Number` y el contenido de la cadena no es un número, se va a producir un fallo en tiempo de ejecución, lo cual, dependiendo del modo en el que se haya hecho la conversión, puede no producir un número real (por ejemplo, `Number` en caso de fallo devuelve `NaN`).

7.2.2. Conversión de cadenas

Además, se pueden realizar conversiones de otros tipos a cadena, utilizando la llamada `String()`. En el ejemplo mostrado a continuación, se muestra la conversión de tipo booleano a cadena, pero se pueden realizar cualquier tipo de conversión.

```
let valor = true;
alert(typeof valor); // boolean

valor = String(valor); // ahora valor es una cadena "true"
alert(typeof value); // string
```

8. Operaciones básicas

Esta sección presenta las principales operaciones soportadas por *JavaScript*. Entre ellas, cabe destacar las operaciones `aritméticas` (sección 8.1), las operaciones con `cadenas` (sección 8.2) y otras operaciones como `modificación inplace` e `incremento y decremento` (secciones 8.3 y 8.4).

8.1. Operaciones aritméticas

Las operaciones aritméticas son ampliamente utilizadas en los lenguajes de programación. *JavaScript* soporta las siguientes operaciones: `suma`, `resta`, `división`, `multiplicación`, `módulo` y `exponenciación`. A continuación se muestra un ejemplo de cada una de las operaciones:

Listado 4: Ejemplos de operaciones aritmeticas en JavaScript

```
alert( 5 + 2 ); // 7
alert( 5 - 2 ); // 3,
alert( 8 / 4 ); // 2
alert( 5 * 4 ); // 20
alert( 5 % 2 ); // 1, el resto de dividir 5 entre 2
alert( 2 ** 3 ); // 8, elevar 2 al cubo
```

8.2. Concatenación de cadenas

Además, se permiten las operaciones de concatenación de cadenas. Basta con utilizar el operador + para concatenar dos cadenas dadas:

```
let s = "my" + "string";
alert(s); // mystring
```

Cabe destacar que se pueden concatenar cadenas y números, sin necesidad especial de hacer conversiones intermedias:

```
alert( '1' + 2 ); // "12"
alert( 2 + '1' ); // "21"
```

Pero se debe tener especial cuidado con la predecencia de operadores, ya que en el siguiente ejemplo, el lenguaje suma los dos primeros números antes de concatenarlos con la tercera cadena:

```
alert(2 + 2 + '1' ); // "41" y no "221"
```

8.3. Modificación 'inplace'

El lenguaje permite además atajos del estilo de la modificación `inplace`. De esta forma se pueden utilizar los operadores `+=` y `*=` para realizar operaciones compactando espacio:

```
let n = 2;
n += 5; // n = 7 (igual que n = n + 5)
```

```
n *= 2; // ahora n = 14 (igual que n = n * 2)

alert( n ); // 14
```

8.4. Incremento/decremento

A su vez, *JavaScript* permite el uso de los operadores incremento (++) y decremento (−−) para compactar las sentencias y aumentar la legibilidad de los programas. El primero incrementa en 1 la cantidad almacenada en la variable, mientras que el segundo lo decrementa.

```
let contador = 1;
contador ++;
alert(contador) // 2

contador--;
alert(contador) // 1
```

9. Estructuras condicionales

Un aspecto muy importante en los lenguajes de programación son las sentencias condicionales, aquellas que permiten realizar un conjunto de operaciones cuando se cumple una condición y otro conjunto distinto de operaciones cuando no se cumplen.

9.1. If-then-else

La estructura por excelencia es `if-then-else`, que como no podría ser de otra forma, está soportado en *JavaScript*. Tal como se puede observar en el listado posterior, se evalúa una condición con la sentencia `if`, si se cumple se ejecuta el código inmediatamente posterior englobado entre llaves. Si por el caso contrario no se cumple, se ejecuta el código seguido de la sentencia `else`.

```
if (condicion es verdadera) {
   // el codigo se ejecuta
} else {
   // el codigo se ejecuta
}
```

En el listado mostrado a continuación se puede observar un ejemplo concreto. Se evalua si la constante edad es mayor o igual que 18 y dependiendo de su valor se muestra por pantalla un mensaje u otro.

```
const edad = 21;
if (edad >= 18) {
  alert("La edad corresponde a una persona mayor de edad.");
} else {
  alert("La edad corresponde a una persona menor de edad.");
}
```

Cabe destacar que se pueden anidar instrucciones `if` a través de la directiva `else if`. A continuación se muestra un ejemplo de su uso:

```
const edad = 21;
if (edad < 18) {
  alert("La edad corresponde a una persona menor de edad.");
} else if (edad == 19 && edad <= 21) {
   alert("La edad corresponde a una persona mayor de edad, salvo en
      ↪ en algunos paises y estados.");
}else {
  alert("La edad corresponde a una persona mayor de edad.");
}
```

9.2. Operadores lógicos

A la hora de utilizar estructuras condicionales, es necesario construir convenientemente las sentencias a evaluar. Para ello, se utilizan los operadores: `AND`, `OR`, `NOT`.

En el caso que queramos que dos sentencias condicionales se cumplan, se debe utilizar el operador `AND` que se escribe con los símbolos &&:

```
if(A&&B)
```

Con un ejemplo más concreto y siguiendo la línea del anterior:

```
const edad = 21;
var esEspanyol=true;

if (edad < 18 && esEspanyol) {
  alert("La edad corresponde a una persona menor de edad.");
} else if (edad >= 18 && esEspanyol) {
   alert("La edad corresponde a una persona mayor de edad.");
}
```

Si se quiere comprobar que, dadas dos o más sentencias, al menos una es correcta, se utiliza el operador `OR` a través de los símbolos ||:

```
if(A||B)
```

A continuación se puede observar cómo se comprueba la mayoría de edad si la variable es mayor que 21, o es mayor o igual que 18 y es español, o es mayor o igual que 16 y además es británico.

```
const edad = 21;
var esBritanico=true
var esEspanyol=false;

if (edad > 21 || (edad >=18 && esEspanyol) || (edad >=16 &&
   ↪ esBritanico)) {
  alert("La edad corresponde a una persona mayor de edad.");
} else{
   alert("La edad corresponde a una persona menor de edad.");
}
```

Finalmente, para comprobar que no se cumple una sentencia se utiliza el operador `NOT`, a través de su codificación !:

```
if(!A)
```

En el siguiente caso se comprueba que, dada una edad, en el caso que sea menor que 16 o que la edad esté comprendida entre 16 y 17 sin ser británico entonces la persona es menor de edad. En caso que la edad sea mayor que 18 o mayor que 16 y británico, entonces es mayor de edad.

```
const edad = 16;
var esBritanico=true

if ((edad <16 )|| (edad >=16 && edad <18 && !esBritanico)) {
  alert("La edad corresponde a una persona menor de edad.");
} else {
   alert("La edad corresponde a una persona mayor de edad.");
}
```

10. Bucles

En numerosas ocasiones, nos encontramos ante la casuística de, a la hora de desarrollar un programa, tener que repetir una instrucción o grupo de instrucciones múltiples veces. De hecho, hay situaciones donde no se sabe de ante mano cuántas veces se debe repetir un fragmento de código, como por ejemplo la búsqueda de un elemento. Con el objetivo de reducir espacio y poder aplicar el fragmento de código de manera general, es necesario el uso de `bucles`. El siguiente listado muestra los tres tipos de bucles más utilizados en *JavaScript*: `while` (sección 10.1), `do-while` (sección 10.2) y `for` (sección 10.3).

Listado 5: Tres mecanismos distintos para implementar bucles en *JavaScript*: `while`, `do-while` y `for`.

```
// Bucle while
```

```
while (condicion) {
  ...
}

// Bucle do-while
do {
  ...
} while (condicion);

// Bucle for
for (inicio; condicion; paso) {
  // ... loop body ...
}
```

10.1. Bucle while

En primer lugar el bucle `while`, evalua una condición y repite el fragmento de código definido directamente a continuación, hasta que se deja de cumplir la condición evaluada inicialmente. En el siguiente ejemplo, se repite el mensaje 'El numero es X' 10 veces, donde X toma los valores de 0 a 9 a lo largo de las iteraciones del bucle.

```
i=0;
while (i < 10) {
  text = "El numero es " + i;
  alert(text);
  i++;
}
```

Cabe destacar que este tipo de bucles se utiliza para realizar `búsquedas`, quiere decir, cuando el número de veces que se va a iterar el bucle es indefinida. Un caso específico de búsqueda, puede ser encontrar un número dentro de un vector.

```
let i = 0, valor = 9;
let bEncontrado=false;
```

```
let array = [7,5,6,8,9];

while(i <= array.length && !bEncontrado) {
    if (array[i] == valor)
        bEncontrado= true;
    i++;
}
```

10.2. Bucle do-while

De forma similar al caso anterior, el bucle `do-while` permite repetir un fragmento de código una cantidad indeterminada de ocasiones. La única diferencia radica en que la comprobación de la sentencia condicional se realiza al final del bucle, lo que nos permite asegurarnos la ejecución, al menos una vez, del código englobado en el bucle.

```
i=0;
do {
  text = "El numero es " + i;
  alert(text);
  i++;
}while (i < 10);
```

De manera análoga los bucles `while`, se utilizan en `búsquedas`.

10.3. Bucle for

Por último, el bucle for se utiliza cuando se conoce el número de iteraciones que se quiere realizar. Para ello hay que proporcionar una inicialización, una condición y una operación que se realiza en cada iteración, la cual nos permite controlar el número de iteraciones completas del bucle.

En el siguiente ejemplo, la inicialización corresponde a la variable $i = 0$, la condición $i < 10$ y la operación la incrementación de la variable inicializada $i++$.

```
i=0;
for(i=0;i<10;i++) {
  text = "El numero es " + i;
  alert(text);
  i++;
}
```

Este tipo de bucles se utiliza en `recorridos`, cuando el número de iteraciones es conocido con anterioridad. Por ejemplo, se utiliza cuando se quiere encontrar el elemento más grande de un vector:

```
let mayor = -1;
let array = [7,5,6,8,9];

for (let i = 1; i < array.length; i++) {
    if (array[i] > mayor) {
        mayor = array[i];
    }
}
```

11. Funciones

En programación, es habitual que se requiera la utilización de parte del código programado en distintas partes del programa. Para maximizar la mantenibilidad de los programas y reducir la cantidad de espacio requerido, se utilizan `funciones`.

11.1. Declaración y uso

Estas funciones tienen como objetivo principal encapsular el código que necesita ser utilizado en distintas partes del programa, maximizando su reutilización. Las funciones comienzan con la palabra reservada `function`, seguido de un nombre

de la función, y entre paréntesis, un conjunto de parámetros separados por comas. Finalmente entre llaves, se ubica el código de la función:

```
function nombre(parametro1, parametro2, ..., parametroN) {
 // body
}
```

Por ejemplo, a continuación se muestra una función, la cual recibe tres parámetros y muestra por pantalla un mensaje. Esta función es invocable desde distintas partes del programa.

```
function mostrarMensaje(remitente, destino, texto) {
  alert( "Mandando mensaje de " + remitente + ': ' + texto );
}

let remitente = "Paco", destino = "Flores", texto = "Hola!";
mostrarMensaje(remitente, destino, texto);
```

11.2. Funciones anónimas

En los últimos años, se ha extendido el uso de funciones anónimas (también llamadas *arrow functions*). Estas funciones no tienen nombre y tienen como objetivo agilizar el desarrollo de software proporcionando atajos. Tienen la siguiente sintaxis:

```
let func = (arg1, arg2, ..., argN) => expression;
```

Como se puede observar, se asignan a una variable y se invocan con el nombre de esta variable.

```
let sum = (a, b) => a + b;

/* Version corta de:
function sum(a, b) {
  return a + b;
}
```

```
*/

alert(sum(1, 2)); // 3
```

Cabe destacar que este tipo de funciones se puede invocar de distintas maneras:

Listado 6:

```
// 1
() => expression
// 2
param => expression
// 3
(param) => expression
// 4
(param1, paramN) => expression
// 5
() => {
  //body
}
// 6
param => {
  //body
}
// 7
(param1, paramN) => {
  //body
}
```